

D-MIND: Disaster Management and Intelligent Navigation Drone

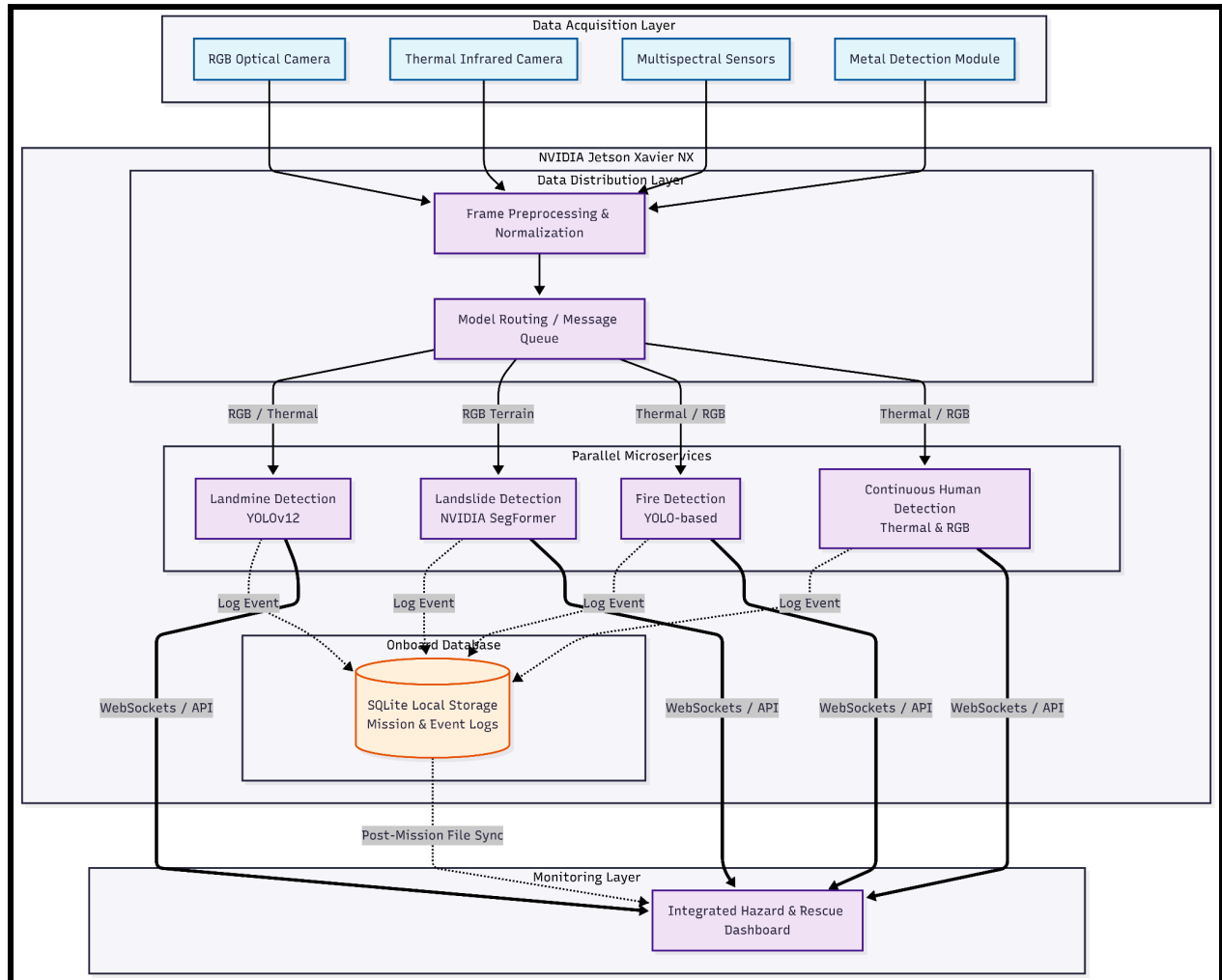


Fig 1: Architecture Diagram for DMIND

Architecture: Hybrid Event-Driven Microservices on an Edge AI platform

This was the natural choice for D-MIND. The system needs to run multiple independent detection pipelines simultaneously, aggregate their outputs into a single dashboard, and do all of this in real time often with no internet connection. Microservices handle the modularity; event driven communication handles the speed.

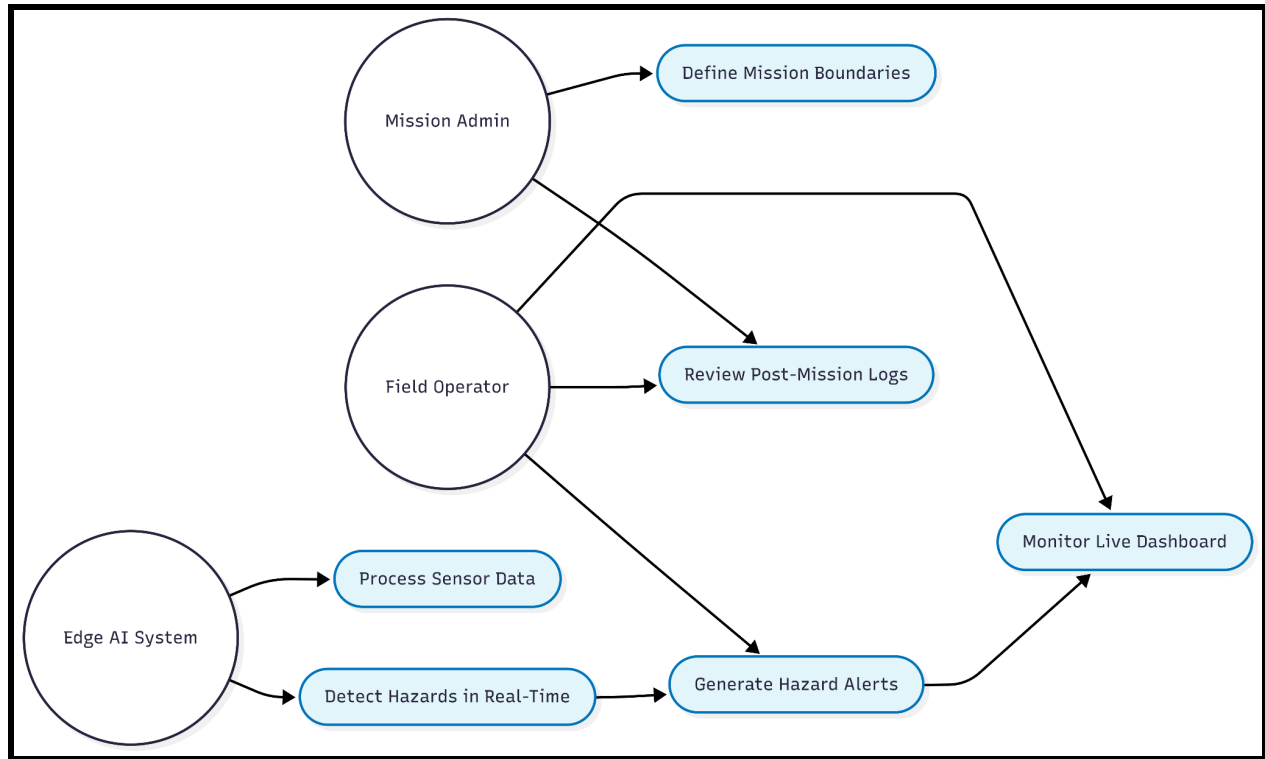


Fig 1.1 : Dataflow Diagram for DMIND

1. Application

1.1 Microservices

Each functional component runs as its own independent service. If one fails or needs updating, the rest keep running.

1.1.1 Data Acquisition Service — manages the UAV and pulls data from the onboard sensors: RGB camera, thermal infrared camera, multispectral sensors, and metal detection module.

1.1.2 Data Distribution Service — the routing layer. It takes incoming frames and sensor data, preprocesses and normalizes them, and sends them to the right AI modules.

1.1.3 AI Processing Modules — four independent services running in parallel:

1.1.3.1 Landmine Detection — YOLOv12, outputs bounding boxes and confidence scores for surface and partially buried mines.

1.1.3.2 Landslide Detection — NVIDIA SegFormer, outputs segmentation masks and terrain classification maps.

1.1.3.3 Fire Detection — YOLO-based model analyzing thermal imagery and smoke features, outputs hotspot locations and smoke plume alerts.

1.1.3.4 Human Detection — runs continuously on thermal and RGB feeds, outputs body bounding boxes and rescue priority markers.

1.1.4 Dashboard Service — aggregates all detection outputs and renders them on the live operator map.

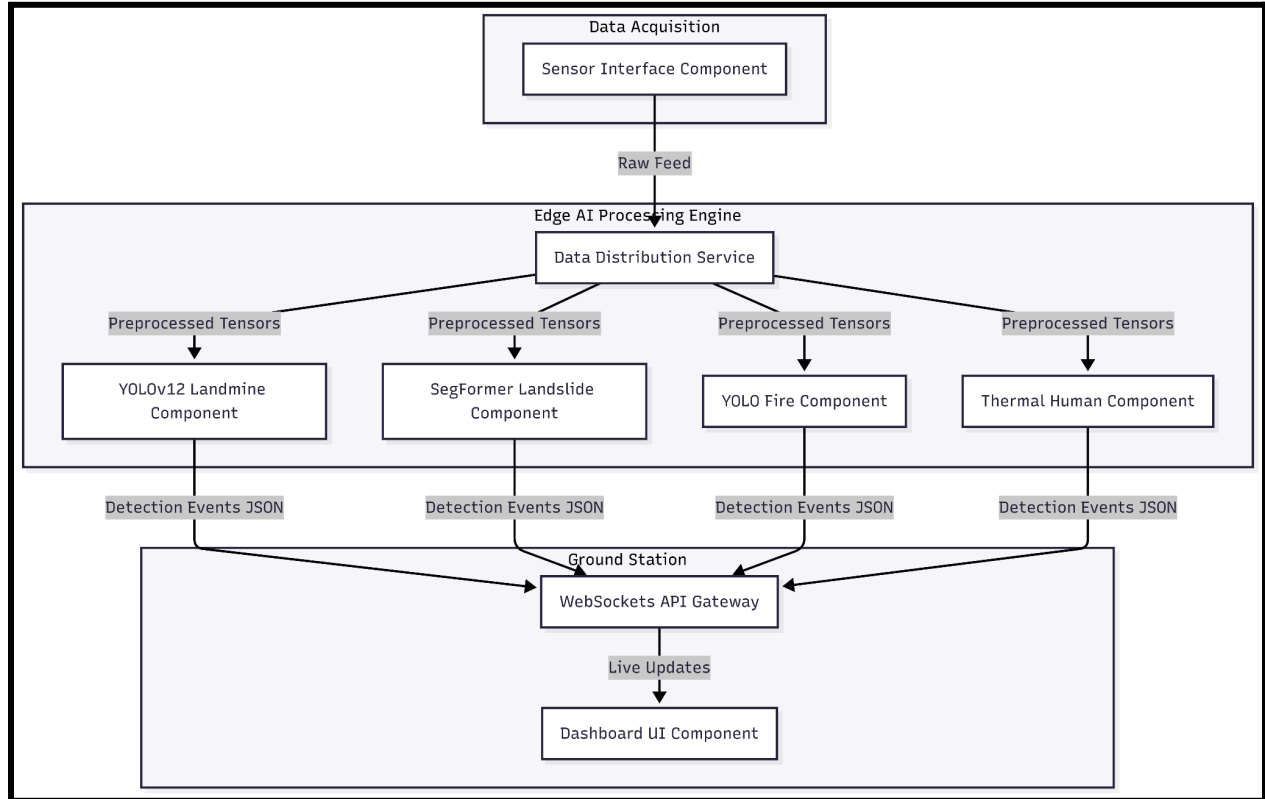


Fig 2: Component Diagram for DMIND

1.2 Event-Driven Flow

The system is built around events rather than polling, which is what makes real-time parallel processing possible.

1.2.1 The drone captures imagery and pushes it as a continuous data stream to the distribution layer.

1.2.2 The distribution layer routes each frame to the relevant modules thermal frames go to both Fire Detection and Human Detection simultaneously, for example.

1.2.3 Each AI module emits a detection event when it finds something (e.g., "landmine detected, 95% confidence"). The dashboard subscribes to these events and updates the hazard map the moment they arrive.

1.3 Why Not Serverless

Serverless works well for cloud-based scalability. It does not work for a drone operating in a disaster zone with no reliable internet connection.

D-MIND runs entirely on the **NVIDIA Jetson Xavier NX** onboard the aircraft. Models are optimized with TensorRT and FP16 quantization so inference is fast enough to be useful in the field with no cloud dependency, no latency from a round trip to a remote server.

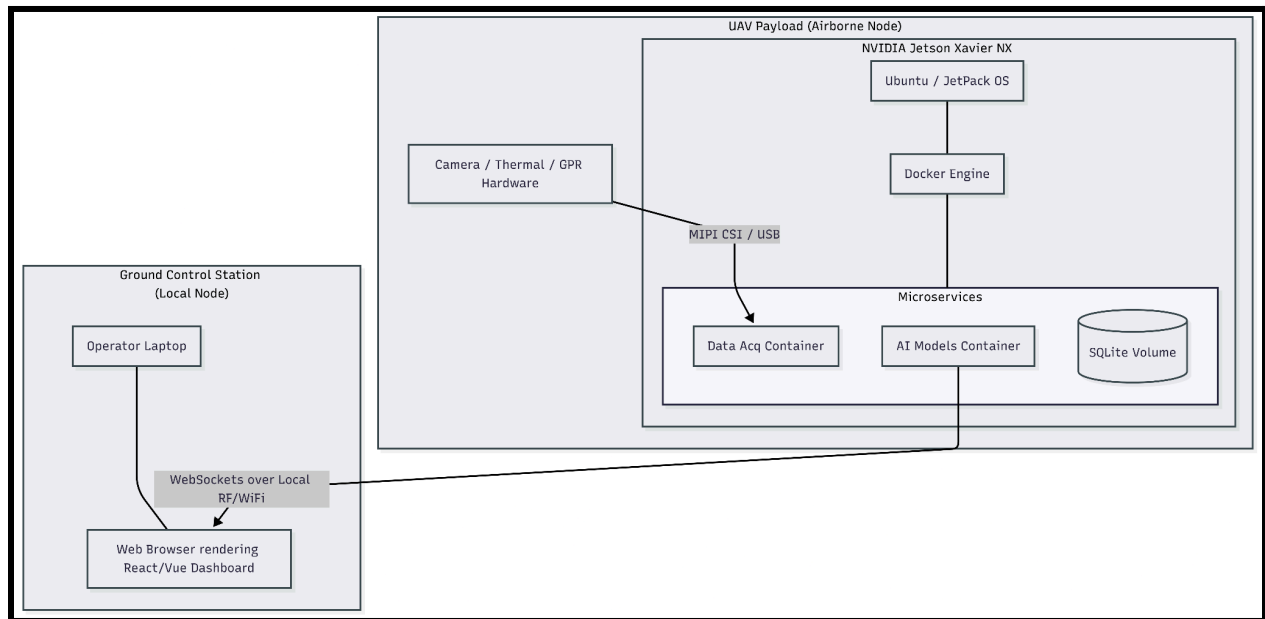


Fig 3 : Deployment Diagram for DMIND

2. Database

Processing happens on the edge, but the system still needs to log what it finds. D-MIND uses a lightweight onboard database (SQLite or a time-series equivalent) to record mission events for the live dashboard and for post-mission analysis.

2.1 Core Entities

Three entities capture everything the system needs to know about a mission:

2.1.1 Mission — the flight record: ID, timestamp, operator, and the geographic boundaries of the search area.

2.1.2 TelemetryLog — the drone's continuous state throughout the flight: GPS coordinates, altitude, battery level.

2.1.3 HazardDetection — the AI outputs: what was detected, how confident the model was, where it was, and a reference to the source image.

2.2 Schema

2.2.1 Mission

2.2.1.1 Mission_ID (PK, UUID)

2.2.1.2 Start_Time (Timestamp)

2.2.1.3 Status (Active / Completed / Aborted)

2.2.2 Telemetry_Log

2.2.2.1 Log_ID (PK, UUID)

2.2.2.2 Mission_ID (FK)

2.2.2.3 Timestamp (Datetime)

2.2.2.4 Latitude (Float)

2.2.2.5 Longitude (Float)

2.2.3 Hazard_Events

2.2.3.1 Event_ID (PK, UUID)

2.2.3.2 Mission_ID (FK)

2.2.3.3 Hazard_Category (Landmine / Landslide / Fire / Human)

2.2.3.4 Confidence (Float, 0.0–1.0)

2.2.3.5 Latitude (Float)

2.2.3.6 Longitude (Float)

2.2.3.7 Bounding_Box_Data (JSON)

2.3 Use Case Diagram

The Use Case Diagram represents the interaction between external actors and the D-MIND system. It illustrates how different users and entities communicate with the system to perform hazard detection, mission monitoring, model management, and rescue support.

The diagram provides a functional view of the system and helps in understanding the services offered by the system from the user's perspective.

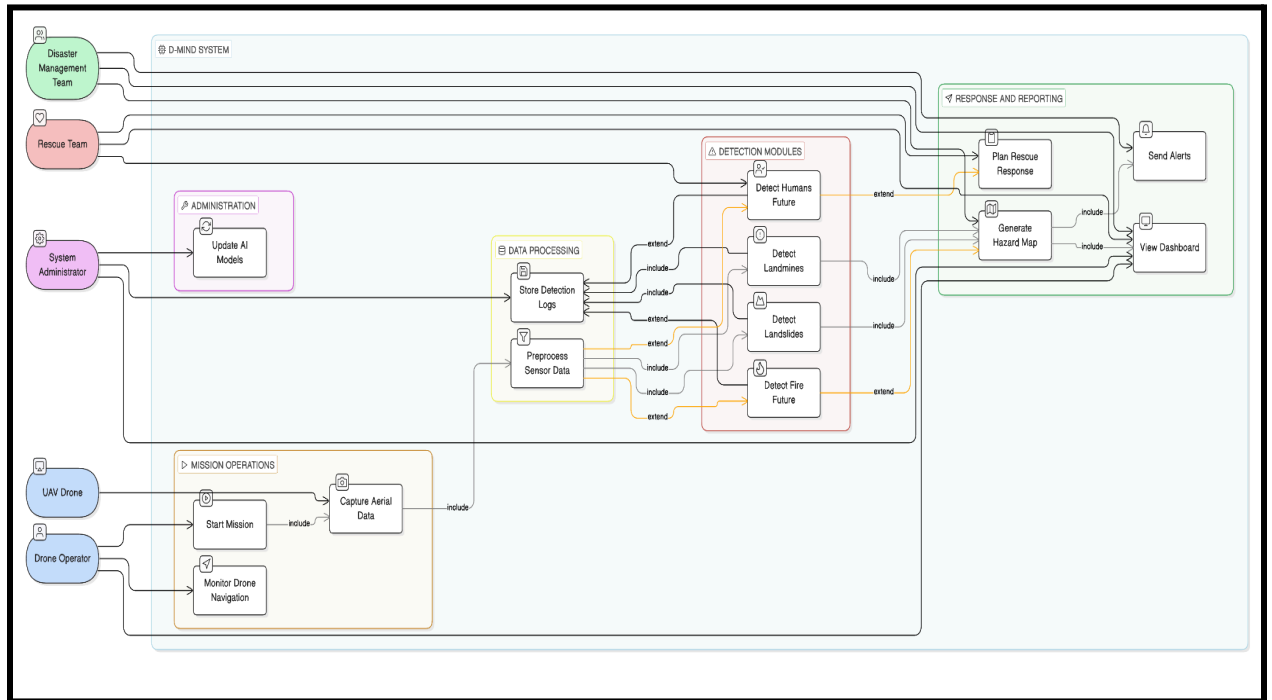


Fig 4 : Use Case Diagram for DMIND

2.3.1 Actors in the System

2.3.1.1 Drone Operator

The Drone Operator is responsible for initiating the mission, monitoring the UAV movement, and viewing hazard detection results on the dashboard.

2.3.1.2 UAV / Drone

The UAV acts as the data acquisition entity. It captures aerial images and sensor data from the target region and sends them to the system for processing.

2.3.1.3 Disaster Management Team

The Disaster Management Team uses the system outputs to study hazard maps, receive alerts, and support decision-making during disaster response.

2.3.1.4 Rescue Team

The Rescue Team uses the outputs of the system, especially human detection and rescue planning results, to carry out rescue operations in hazardous areas.

2.3.1.5 System Administrator

The System Administrator manages datasets, updates AI models, supervises system logs, and maintains the dashboard and detection framework.

2.3.2 Main Use Cases

2.3.2.1 Start Mission

This use case allows the Drone Operator to initiate a disaster monitoring or hazard detection mission using the UAV.

2.3.2.2 Capture Aerial Data

The UAV captures RGB, thermal, and multispectral imagery from the target region for further analysis.

2.3.2.3 Preprocess Sensor Data

Captured sensor data is normalized, resized, and prepared before being passed into the AI detection pipelines.

2.3.2.4 Detect Landmines

The system uses the YOLOv12 model to identify and localize landmines from aerial images.

2.3.2.5 Detect Landslides

The system uses the SegFormer model to generate pixel-level segmentation masks for landslide-affected regions.

2.3.2.6 Detect Fire (Future)

This future use case extends the system to detect wildfire or thermal hotspot regions from aerial and thermal imagery.

2.3.2.7 Detect Humans (Future)

This future module enables the system to identify trapped or stranded humans in disaster zones to assist rescue teams.

2.3.2.8 Generate Hazard Map

The system combines outputs from different detection modules to create an integrated hazard map.

2.3.2.9 Send Alerts

The system generates warning alerts whenever a hazard is detected with sufficient confidence.

2.3.2.10 View Dashboard

Users such as the Drone Operator, Disaster Management Team, Rescue Team, and Administrator can visualize mission outputs and hazard overlays through the dashboard.

2.3.2.11 Plan Rescue Response

This use case helps the Rescue Team identify rescue routes and prioritize interventions based on hazard and human detection outputs.

2.3.2.12 Update AI Models

The System Administrator updates or retrains AI models for improved performance and future deployment.

2.3.2.13 Store Detection Logs

The system stores detection outputs and mission records for future reference, analytics, and evaluation.

2.3.2.14 Monitor Drone Navigation

The Drone Operator monitors the movement, direction, and coverage path of the drone during mission execution.

2.3.3 Relationships in the Diagram

2.3.3.1 The **Drone Operator** is associated with:

2.3.3.1.1 Start Mission

2.3.3.1.2 View Dashboard

2.3.3.1.3 Monitor Drone Navigation

2.3.3.2 The **UAV / Drone** is associated with:

2.3.3.2.1 Capture Aerial Data

2.3.3.3 The **Disaster Management Team** is associated with:

2.3.3.3.1 View Dashboard

2.3.3.3.2 Generate Hazard Map

2.3.3.3.3 Send Alerts

2.3.3.4 The **Rescue Team** is associated with:

2.3.3.4.1 Detect Humans

2.3.3.4.2 Plan Rescue Response

2.3.3.4.3 View Dashboard

2.3.3.5 The **System Administrator** is associated with:

2.3.3.5.1 Update AI Models

2.3.3.5.2 Store Detection Logs

2.3.3.5.3 View Dashboard

2.3.4 Include and Extend Relationships

2.3.4.1 Include Relationships

The following use cases are mandatory and directly form part of the system workflow:

2.3.4.1.1 Start Mission includes Capture Aerial Data

2.3.4.1.2 Capture Aerial Data includes Preprocess Sensor Data

2.3.4.1.3 Preprocess Sensor Data includes Detect Landmines

2.3.4.1.4 Preprocess Sensor Data includes Detect Landslides

2.3.4.1.5 Detect Landmines includes Generate Hazard Map

2.3.4.1.6 Detect Landslides includes Generate Hazard Map

2.3.4.1.7 Generate Hazard Map includes Send Alerts

2.3.4.1.8 Generate Hazard Map includes View Dashboard

2.3.5 Extend Relationships

The following use cases are future or optional extensions:

2.3.5.1 Preprocess Sensor Data extends Detect Fire

2.3.5.2 Preprocess Sensor Data extends Detect Humans

2.3.5.3 Detect Humans extends Plan Rescue Response

2.3.5.4 Detect Fire extends Generate Hazard Map

2.3.6 Significance of the Use Case Diagram

The Use Case Diagram is important because it:

2.3.6.1 identifies all external users of the system

2.3.6.2 clarifies the major services provided by D-MIND

2.3.6.3 shows the boundary between the users and the internal system

2.3.6.4 highlights future system extendibility through fire and human detection modules

2.3.6.5 provides a high-level functional understanding of the complete architecture

2.4 Class Diagram

The Class Diagram represents the static structure of the D-MIND system. It shows the major software classes, their attributes, methods, and relationships. It explains how drone components, AI models, mission control, hazard outputs, and administrative modules are organized inside the system.

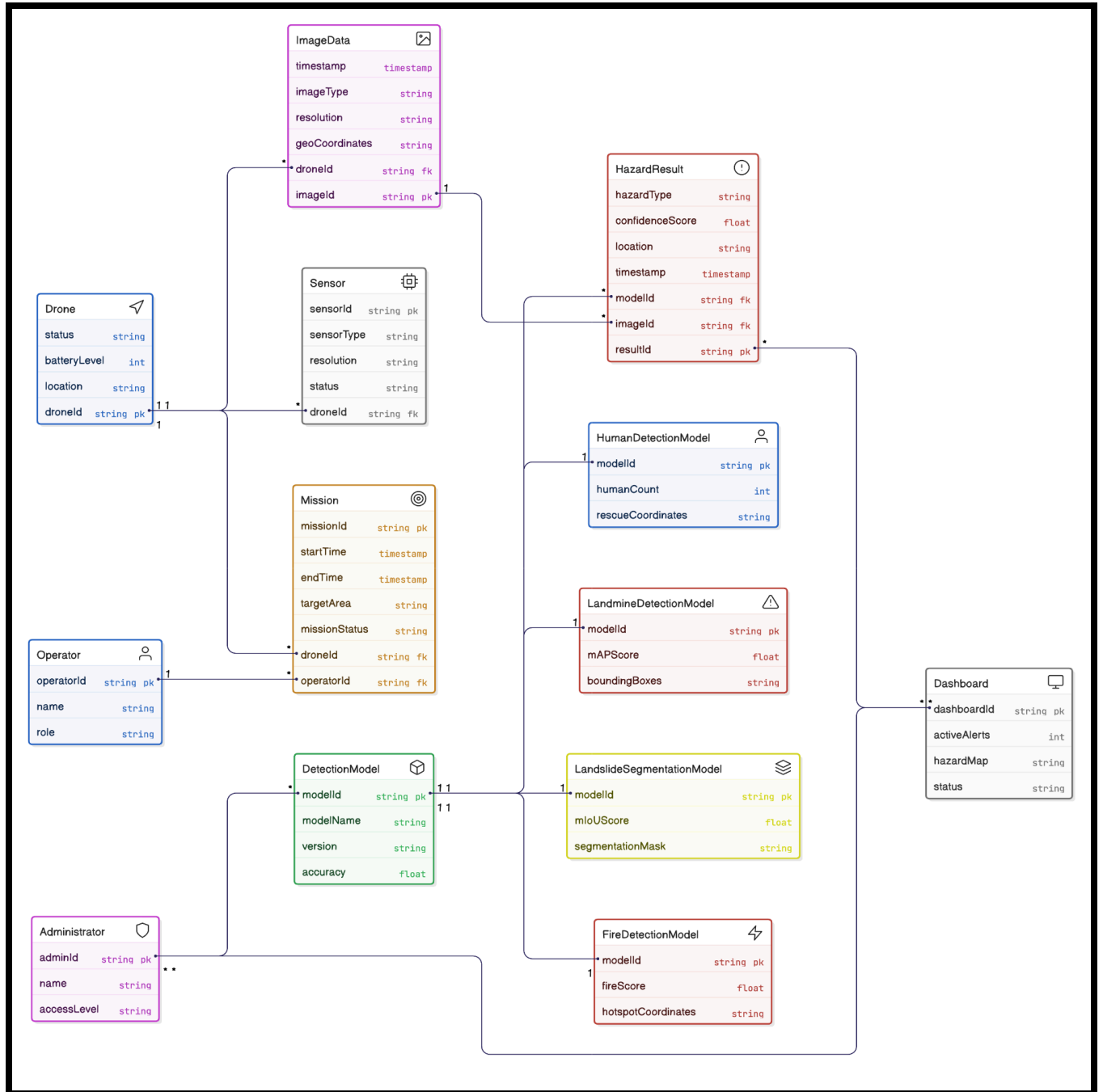


Fig 5 : Class Diagram for DMIND

3. Data Exchange Contract

3.1 Frequency

3.1.1 Sensor → Distribution: Continuous, 30–60 frames per second.

3.1.2 Distribution → AI Modules: Real-time parallel dispatch, rate-limited by each module's processing speed.

3.1.3 AI Modules → Dashboard: Event-triggered. Data only moves when a detection crosses the confidence threshold, no unnecessary traffic.

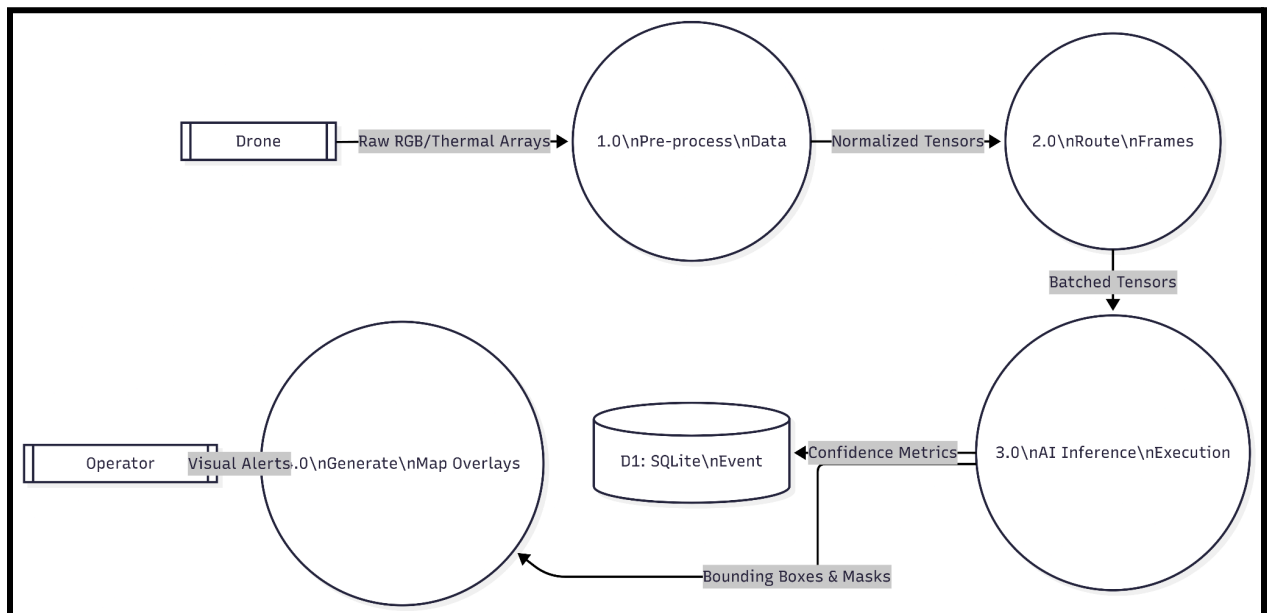


Fig 6.1 : Dataflow Diagram for DMIND

3.2 Data Types

3.2.1 Raw input - unprocessed RGB arrays, thermal heatmaps, multispectral tensors.

3.2.2 Preprocessed data - resized and normalized image tensors, ready for YOLOv12 or SegFormer.

3.2.3 Inference outputs - bounding box coordinates (X, Y, W, H), segmentation masks, hazard labels, confidence scores.

3.2.4 Telemetry - continuous GPS coordinates mapped to each detection so operators get exact geographic markers, not just relative positions.

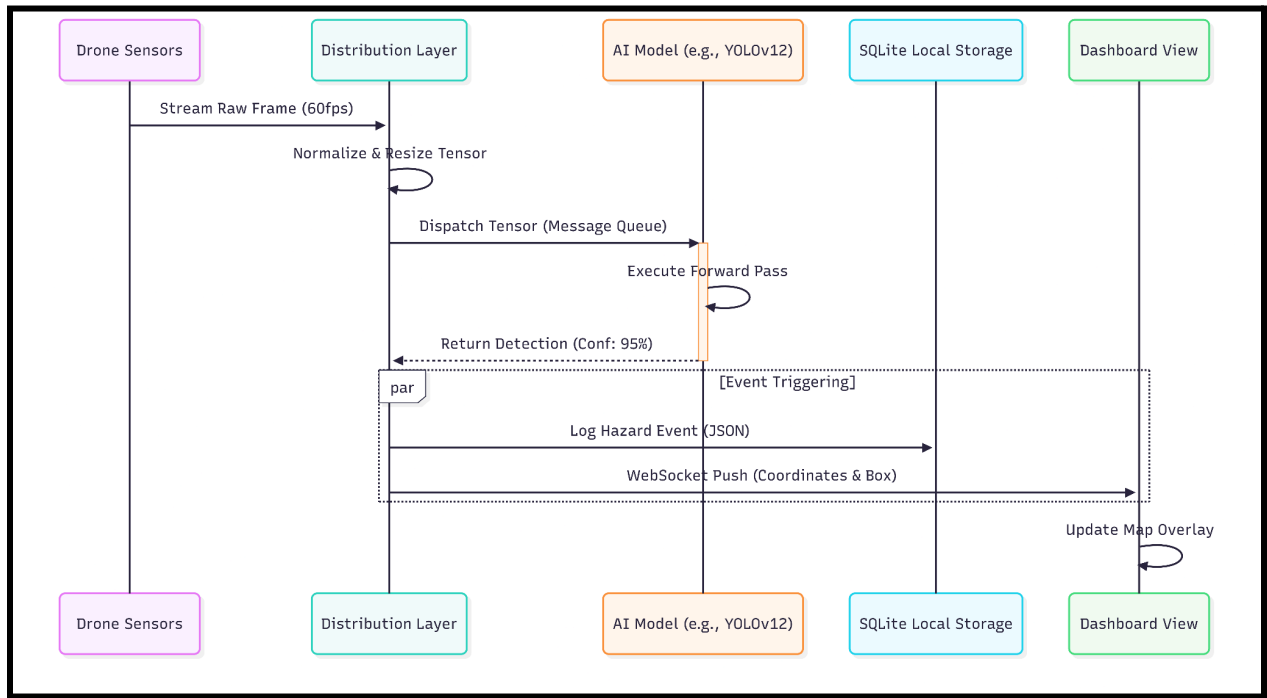


Fig 6.2 : Sequence Diagram for DMIND

3.3 ER Diagram

The ER Diagram represents the database structure of the D-MIND system. It defines the entities involved in mission management, drone operations, image storage, AI model processing, detection outputs, and alert generation. It also shows the relationships between these entities and supports efficient storage and retrieval of disaster monitoring data.

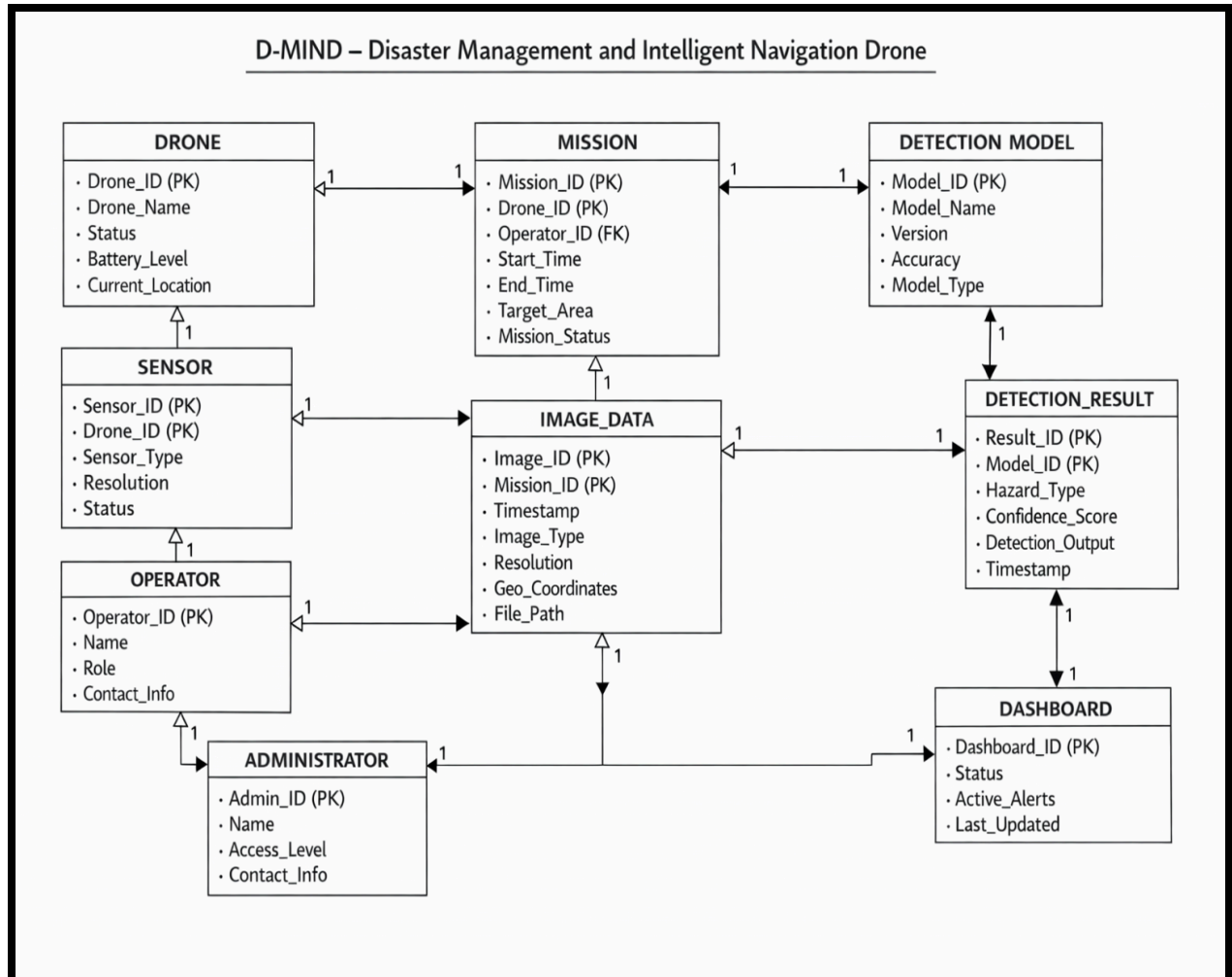


Fig 6.3 : ER Diagram for DMIND

3.4 How Data Moves

3.4.1 Message queues (pub/sub) - used internally on the Jetson Xavier NX (via ROS topics, MQTT, or ZeroMQ) to route frames from the distribution layer to the AI modules. This prevents any single module from becoming a bottleneck and keeps processing genuinely parallel.

3.4.2 WebSockets / API - used to push final detection events and live video from the edge device to the operator dashboard. Low latency, persistent connection.

3.4.3 File - used post-mission. Raw images, segmentation masks, and the SQLite log are saved to local storage for after-flight review by data analysts and disaster management teams.